

Student

I use `with open('file.txt') as f` all the time, but I honestly have no idea what's happening under the hood. Is it just syntactic sugar for try/finally?

ChatGPT • AI Assistant

CONCEPT CARD

Almost — but more elegant than that. The `with` statement implements Python's **context management protocol**. It guarantees that setup happens before a block of code runs, and cleanup happens after — whether the block exits normally, via `return`, or via an exception. The two methods that make this work are `__enter__` (called at the top of the `with` block) and `__exit__` (called at the bottom, always, no matter what). Think of it as a contract: 'I promise to clean up after myself.'

Student

What kinds of resources need this kind of protection?

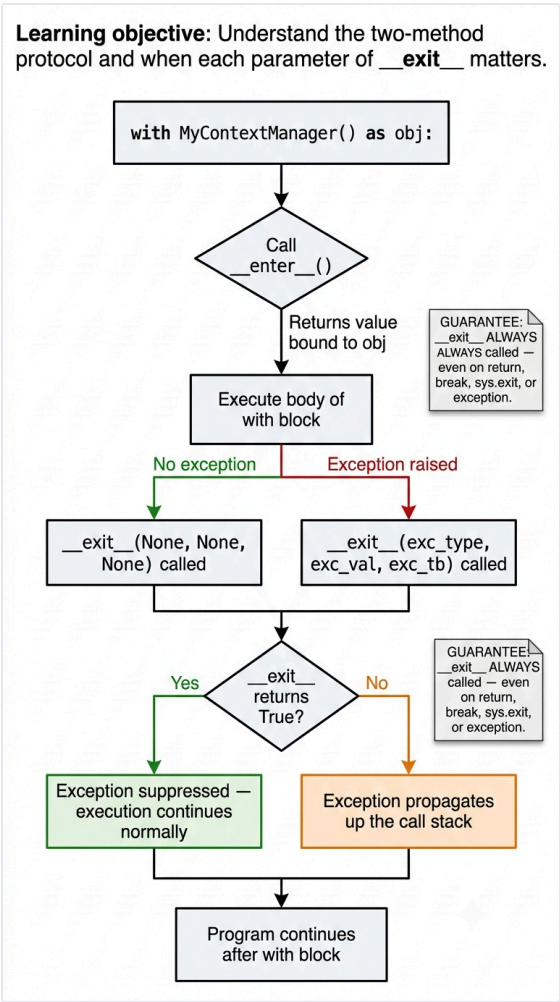
ChatGPT • AI Assistant

Any resource that must be released after use:

- **File handles** — must be closed or the OS runs out of file descriptors
- **Database connections** — must be committed or rolled back, then closed
- **Network sockets** — must be closed or you leak connections
- **Locks** — must be released or the program deadlocks
- **Temporary directories** — must be deleted or disk fills up

The guarantee that `__exit__` ALWAYS runs is what makes context managers so powerful. A raw try/finally works too — but context managers package that guarantee into a reusable, composable object.

Learning objective: Understand the two-method protocol and when each parameter of `__exit__` matters.



Context Manager Protocol — The with Statement Execution Flow

Context Manager Protocol — Overview at a Glance

Method	When Called	Parameters	Return Value Matters?
<code>__enter__</code>	At start of with block	self only	Its return value is bound to 'as' variable
<code>__exit__</code> (normal)	After block completes normally	self, None, None, None	Return value ignored
<code>__exit__</code> (exception)	When exception raised in block	self, exc_type, exc_val, exc_tb	True = suppress exception; False/None = propagate
<code>@contextmanager</code> enter	Code before yield	N/A (generator function)	Yielded value bound to 'as' variable

Method	When Called	Parameters	Return Value Matters?
@contextmanager exit	Code after yield (in finally)	N/A — caught by try/except in generator	Raising suppresses; re-raising propagates

THE WITH STATEMENT — `__ENTER__` AND `__EXIT__` PROTOCOL

ChatGPT • AI Assistant

A two-column comparison diagram. Left column (green background, labeled 'Normal Exit — No Exception'): Shows the with block completing cleanly. Arrow points to `__exit__` call box showing: `'__exit__(self, None, None, None)'`. All three parameter boxes below it show 'None' with grey background. Note: 'Return value of `__exit__` is ignored on normal exit.' Right column (red background, labeled 'Exception Raised Inside Block'): Shows a lightning bolt icon at the body block, labeled `'raise ValueError("bad data")'`. Arrow points to `__exit__` call box showing: `'__exit__(self, exc_type, exc_val, exc_tb)'`. Three parameter boxes: `exc_type` box = `'ValueError (the class)'`; `exc_val` box = `'ValueError("bad data") (the instance)'`; `exc_tb` box = `'traceback object (the stack trace)'`. Below, a decision diamond: 'Returns True?' Yes → 'Exception suppressed'. No/None → 'Exception re-raised'. Learning objective: Know exactly what each `__exit__` parameter contains and when suppression applies.

`__exit__` Parameters — None vs Exception Path

Student

Can you show me what `with open('file.txt') as f` actually does — broken down step by step?

ChatGPT • AI Assistant

CONCEPT CARD

- Sure. When Python sees a `with` statement, here is the exact sequence:
- Python evaluates the expression after `with` — e.g., `open('file.txt')` — creating the context manager object
 - Python calls `context_manager.__enter__()` and binds the return value to the `as` variable
 - The body of the `with` block runs

4. Python calls `context_manager.__exit__(exc_type, exc_val, exc_tb)` — whether or not an exception occurred

EXAMPLE CARD

Example 1 — Basic: Peeking inside `open()`'s protocol

python — Code Example

```
# This is roughly what Python does for:
# with open('data.txt') as f:
#     content = f.read()

file_obj = open('data.txt')           # Step 1: create context manager
f = file_obj.__enter__()              # Step 2: __enter__ returns the
file object itself
try:
    content = f.read()                # Step 3: body runs
except Exception as e:
    # Step 4a: exception path
    if not file_obj.__exit__(type(e), e, e.__traceback__):
        raise                        # re-raise if __exit__ returns
False
else:
    # Step 4b: normal path
    file_obj.__exit__(None, None, None)

# The 'with' statement compiles to EXACTLY this logic.
# You never write this manually — the with statement does it for you.
```

Student

So `__enter__` returns the file object, and that's what gets bound to `f` ? The context manager and the resource are the same object here?

ChatGPT • AI Assistant

In the case of `open()`, yes — the file object IS the context manager. `__enter__` returns `self`. But that's not always true. A database connection context manager might return a *cursor* from `__enter__` while the connection object manages the commit/rollback in `__exit__`. What `__enter__` returns and what manages cleanup can be different things.

EXAMPLE CARD

Example 2 — Intermediate: Writing your own class-based context manager

python – Code Example

```

class ManagedFile:
    """
    A class-based context manager that wraps a file.
    Demonstrates __enter__ and __exit__ explicitly.
    """

    def __init__(self, filepath, mode='r'):
        self.filepath = filepath
        self.mode = mode
        self.file = None

    def __enter__(self):
        print(f'[ENTER] Opening {self.filepath}')
        self.file = open(self.filepath, self.mode)
        return self.file    # This is what gets bound to the 'as'
variable

    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f'[EXIT] Closing {self.filepath}')
        if self.file:
            self.file.close()
        if exc_type is not None:
            print(f'[EXIT] Exception occurred: {exc_type.__name__}:
{exc_val}')
        return False    # Do NOT suppress the exception – let it
propagate

# Usage:
with ManagedFile('test.txt', 'w') as f:
    f.write('Hello, context managers!')
    print('Writing... ')
# Output:
# [ENTER] Opening test.txt
# Writing...
# [EXIT] Closing test.txt

# Even if an exception occurs:
try:
    with ManagedFile('test.txt', 'r') as f:
        print(f.read())
        raise ValueError('Something went wrong!')
except ValueError:
    pass

```

```
# Output:
# [ENTER] Opening test.txt
# Hello, context managers!
# [EXIT] Closing test.txt
# [EXIT] Exception occurred: ValueError: Something went wrong!
# (ValueError propagates because __exit__ returned False)
```

Student

What are those three parameters in `__exit__` — `exc_type` , `exc_val` , `exc_tb` ? When do they have values?

ChatGPT • AI Assistant

CONCEPT CARD

The three `__exit__` parameters carry the full exception information:

- `'exc_type'` — the exception *class* (e.g., `ValueError` , `FileNotFoundError`). `None` if no exception.
- `'exc_val'` — the exception *instance* — the actual object with the message and attributes. `None` if no exception.
- `'exc_tb'` — the *traceback* object — shows which line raised the exception. `None` if no exception.

These three together let `__exit__` make an intelligent decision: inspect the exception type and decide whether to suppress it (return `True`) or let it propagate (return `False` or `None`).

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — forgetting that returning `True` suppresses the exception:

python — Code Example

```
def __exit__(self, exc_type, exc_val, exc_tb):
    self.connection.close()
    return True # DANGER: this silently swallows ALL exceptions!
                # Your database error just vanished without a trace
```

✓ Correct — only suppress what you specifically handle:

python — Code Example

```
def __exit__(self, exc_type, exc_val, exc_tb):
    self.connection.close()
    # Return True ONLY if you intentionally handled this exception
    if exc_type is FileNotFoundError:
        print('File not found — handled gracefully')
        return True    # Suppress only FileNotFoundError
    return False       # All other exceptions propagate normally
```

INTERVIEW CARD

- **Frequency:** High — standard Python interview question for mid-level roles
- **Question:** 'What are the three parameters of `__exit__` , and what does returning `True` from `__exit__` do?'
- **Expected Answer:** `exc_type` is the exception class, `exc_val` is the exception instance, `exc_tb` is the traceback. All three are `None` when no exception occurred. Returning `True` suppresses the exception — execution continues after the `with` block as if nothing happened. Returning `False` or `None` allows the exception to propagate.
- **Follow-up:** 'When would you intentionally return `True` from `exit`?'

EXERCISE

Write a class-based context manager `Timer` that:

1. Records the start time in `__enter__`
2. Records the end time in `__exit__`
3. Prints `'Elapsed: X.XXXs'` in `__exit__`
4. Does NOT suppress any exceptions

python — Code Example

```
import time

class Timer:
    def __enter__(self):
        # TODO
        pass

    def __exit__(self, exc_type, exc_val, exc_tb):
        # TODO
        pass

with Timer():
    time.sleep(0.1)
```

```
print('Working ... ')\n# Expected output:\n# Working...\n# Elapsed: 0.100s
```

🔗 PRACTICE QUIZ

Concept Questions:

1. What method is called at the START of a `with` block, and what is its return value used for?
2. In what three situations does `__exit__` get called?
3. What is the difference between `exc_val` and `exc_type` in `__exit__` ?

Output Prediction Questions:

1. What does this print, and does the exception propagate?

python – Code Example

```
class Suppressor:\n    def __enter__(self): return self\n    def __exit__(self, exc_type, exc_val, exc_tb):\n        print(f'Caught: {exc_type}')\n        return True\n\nwith Suppressor():\n    raise ValueError('oops')\nprint('After with block')
```

1. What does this print? Does the file get closed?

python – Code Example

```
with open('nonexistent.txt') as f:\n    data = f.read()\nprint('Done')
```

Debugging Question:

1. A developer writes this context manager for a lock but finds their program deadlocks occasionally:

python – Code Example

```
class LockManager:
    def __init__(self, lock): self.lock = lock
    def __enter__(self):
        self.lock.acquire()
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is None:      # Bug: only releases on
            success!
            self.lock.release()
```

What is the bug and how do you fix it?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. `__enter__` is called at the start. Its return value is bound to the variable after `as` . If there is no `as` clause, the return value is discarded.
2. `__exit__` is called: (a) when the block completes normally, (b) when an exception is raised inside the block, (c) when a `return` or `break` exits the block.
3. `exc_type` is the exception *class* (e.g., `ValueError`). `exc_val` is the exception *instance* — the actual object that has the `.args` and message. You check `exc_type` for the kind; you inspect `exc_val` for the details.
4. Prints `Caught: <class 'ValueError'>` then `After with block` . The exception is suppressed because `__exit__` returns `True` .
5. `FileNotFoundError` is raised by `open()` — BEFORE `__enter__` is called, so `__exit__` is never invoked. The exception propagates and `'Done'` is never printed. The file is never opened, so no cleanup is needed.
6. Bug: the lock is only released when no exception occurs — if an exception happens inside the block, the lock is never released, causing deadlock. Fix: remove the `if exc_type is None` check — always release the lock:
`self.lock.release()` unconditionally, then `return False` .

❓ CHECKPOINT QUESTION

In your own words: why is `__exit__` more powerful than a plain `finally` block for library authors?

CLASS-BASED CONTEXT MANAGER FOR DATABASE CONNECTIONS

ChatGPT • AI Assistant

A flowchart starting at the top with 'with SQLiteConnection(db_path) as cursor:'. From this box, an arrow labeled '__enter__ called' leads to 'conn = connect(); cursor = conn.cursor(); return cursor'. Then 'Body executes SQL operations'. From the body, two paths diverge. Left path (green, labeled 'No exception'): leads to '___exit__(None, None, None)' → 'conn.commit()' → 'conn.close()' → 'Caller continues normally'. Right path (red, labeled 'Exception raised inside body'): leads to '___exit__(exc_type, exc_val, exc_tb)' → 'conn.rollback()' → 'conn.close()' → 'Exception propagates to caller (return False)'. Both paths merge at a box labeled 'Connection closed — no leaks'. A highlighted note: 'ATOMICITY: either ALL operations commit, or NONE do.' Learning objective: Understand why __exit__ is the correct place for commit/rollback logic.

Database Context Manager — Commit vs Rollback Decision Tree

Student

Can we build the database connection example you mentioned? That seems like the most realistic use case I'd encounter in a real job.

ChatGPT • AI Assistant

🌐 REAL WORLD APPLICATION

Absolutely — this is the canonical example for class-based context managers. The rule for database operations is: if the block succeeds, **commit**. If any exception occurs, **rollback**. Either way, **close the connection**. This is exactly the kind of guarantee `__exit__` was designed to provide.

EXAMPLE CARD

Example 1 — Basic: Minimal database context manager skeleton

python — Code Example

```
class DatabaseConnection:
    """
    Context manager that:
    - Opens a DB connection on enter
    - Commits on clean exit
    - Rolls back on any exception
    - Always closes the connection
    """
    def __init__(self, connection_string):
        self.connection_string = connection_string
        self.connection = None
```

```

        self.cursor = None

    def __enter__(self):
        # Simulate connecting (in production: psycopg2.connect(...)
        etc.)
        print(f'[DB] Connecting to {self.connection_string}')
        self.connection = self._connect()
        self.cursor = self.connection.cursor()
        return self.cursor    # Caller gets the cursor, not the
                               connection

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            # An exception occurred – rollback all changes
            print(f'[DB] Rolling back due to {exc_type.__name__}:
{exc_val}')
            self.connection.rollback()
        else:
            # Block completed successfully – commit all changes
            print('[DB] Committing transaction')
            self.connection.commit()
        # Always close the connection
        self.cursor.close()
        self.connection.close()
        print('[DB] Connection closed')
        return False    # Never suppress database exceptions

```

EXAMPLE CARD

Example 2 — Practical: Using the database context manager

python – Code Example

```

# Simulate a DB connection (using sqlite3 which follows DB-API 2.0)
import sqlite3

class SQLiteConnection:
    def __init__(self, db_path):
        self.db_path = db_path
        self.conn = None

    def __enter__(self):
        self.conn = sqlite3.connect(self.db_path)
        return self.conn.cursor()

    def __exit__(self, exc_type, exc_val, exc_tb):

```

```

    if exc_type:
        print(f'[ROLLBACK] {exc_type.__name__}: {exc_val}')
        self.conn.rollback()
    else:
        print('[COMMIT] Transaction committed')
        self.conn.commit()
    self.conn.close()
    return False    # Always propagate DB exceptions

# SUCCESS PATH: both inserts commit together
with SQLiteConnection(':memory:') as cursor:
    cursor.execute('CREATE TABLE users (id INTEGER, name TEXT)')
    cursor.execute("INSERT INTO users VALUES (1, 'Alice')")
    cursor.execute("INSERT INTO users VALUES (2, 'Bob')")
    print('Inserts done')

# Output:
# Inserts done
# [COMMIT] Transaction committed

# FAILURE PATH: exception causes rollback
try:
    with SQLiteConnection(':memory:') as cursor:
        cursor.execute('CREATE TABLE users (id INTEGER, name TEXT)')
        cursor.execute("INSERT INTO users VALUES (1, 'Alice')")
        raise ValueError('Schema validation failed!')    # Simulated
    cursor.execute("INSERT INTO users VALUES (2, 'Bob')")    # Never
    runs
except ValueError:
    pass

# Output:
# [ROLLBACK] ValueError: Schema validation failed!
# (Both inserts are rolled back – Alice was NOT saved)

```

Student

Why is it important that both inserts roll back together? Why not just roll back the last one?

ChatGPT • AI Assistant

This is the concept of **atomicity** — one of the ACID properties of database transactions. Either ALL operations in a block succeed together, or NONE of them do.

If you saved Alice but not Bob, your data is in an inconsistent state — half a batch write succeeded. In financial systems, this is the difference between debiting an account without crediting another. The rollback in `__exit__` undoes the ENTIRE transaction as one unit.

EXAMPLE CARD

Example 3 — Advanced: Nested context managers and multiple resources

python — Code Example

```
import sqlite3
import csv

class SQLiteConnection:
    """(Same as above)"""
    def __init__(self, db_path):
        self.db_path = db_path
        self.conn = None
    def __enter__(self):
        self.conn = sqlite3.connect(self.db_path)
        return self.conn.cursor()
    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type:
            self.conn.rollback()
        else:
            self.conn.commit()
        self.conn.close()
        return False

def import_users_from_csv(csv_path, db_path):
    """
    Read users from CSV and insert into DB.
    Both resources (file + DB) are managed safely.
    """
    with open(csv_path, newline='') as csvfile, \
        SQLiteConnection(db_path) as cursor:    # Both context
managers active!
        reader = csv.DictReader(csvfile)
        for row in reader:
            cursor.execute(
                'INSERT INTO users (id, name, email) VALUES (?, ?,
?)',
                (row['id'], row['name'], row['email'])
            )
        # When this block exits:
        # 1. If successful: DB commits, file closes
```

```
# 2. If any row fails: DB rolls back ALL rows, file still closes
# Both __exit__ methods are called in REVERSE order (DB first,
then file)
```

```
import_users_from_csv('users.csv', 'app.db')
```

Student

You can stack two context managers on the same `with` statement with a comma?

ChatGPT • AI Assistant

Yes! Python 2.7+ supports `with A as a, B as b:` which is equivalent to nested `with` blocks. Both `__exit__` methods are called in **reverse order** — last-opened, first-closed. This follows the same principle as a stack.

⚠ COMMON PITFALL / MISTAKE

✗ Incorrect — committing inside the body, bypassing rollback:

python – Code Example

```
with SQLiteConnection(db_path) as cursor:
    cursor.execute('INSERT INTO users VALUES ...')
    connection.commit()    # Manual commit inside body!
# If an exception occurs AFTER this commit, __exit__ rolls back
# but the data was ALREADY committed – rollback does nothing!
# You've lost atomic guarantees.
```

✓ Correct — let **exit** handle commit/rollback exclusively:

python – Code Example

```
with SQLiteConnection(db_path) as cursor:
    cursor.execute('INSERT INTO users VALUES ...')
    # No manual commits – __exit__ decides
# __exit__ commits on success, rolls back on any exception
```

🧑 INTERVIEW CARD

- **Frequency:** High in backend and data engineering interviews

- **Question:** 'Implement a context manager for a database connection that commits on success and rolls back on any exception.'
- **Expected Answer:** Class with `__enter__` opening connection, `__exit__` checking `exc_type is not None` to decide rollback vs commit, always closing connection, always returning `False` to propagate exceptions.
- **Follow-up:** 'What happens if the commit itself raises an exception inside `__exit__` ?'

🔥 EXERCISE

Extend `SQLiteConnection` to also:

1. Accept a `timeout` parameter for the connection
2. Log the number of rows affected after each successful commit using `cursor.rowcount`
3. If an `IntegrityError` (duplicate key) occurs, log a warning but still rollback and propagate

🔍 PRACTICE QUIZ

Concept Questions:

1. Why does the database context manager return `cursor` from `__enter__` instead of `self` ?
2. Why should `return False` (not `return True`) almost always be used in a database context manager's `__exit__` ?
3. In what order are `__exit__` methods called when two context managers are stacked with a comma?

Output Prediction Questions:

1. Given `SQLiteConnection.__exit__` returns `False` , what happens to a `sqlite3.IntegrityError` raised inside the `with` block?
2. If `conn.commit()` raises an exception inside `__exit__` , what happens to that exception?

Debugging Question:

1. Find the bug — this context manager leaks connections:

python – Code Example

```
class DBConn:
    def __enter__(self):
        self.conn = sqlite3.connect('app.db')
```

```

        self.cursor = self.conn.cursor()
    return self.cursor
def __exit__(self, exc_type, exc_val, exc_tb):
    if exc_type is None:
        self.conn.commit()
        self.conn.close()
    else:
        self.conn.rollback()
        # Bug: forgot to close on the exception path!

```

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Because the *caller* needs to execute SQL — they need the cursor. The connection object is an internal implementation detail that the caller should not directly manipulate.
2. Database exceptions (constraint violations, connection drops) are important signals that callers must handle. Suppressing them would hide data integrity problems silently.
3. Reverse order — last context manager opened is the first to have its `__exit__` called. Like a stack: LIFO (Last In, First Out).
4. The exception propagates after `__exit__` finishes — `return False` means do not suppress.
5. That new exception raised in `__exit__` replaces the original exception and propagates. The original exception is attached as `__context__`. This is a common trap — always wrap the commit in a try/except inside `__exit__` if you need to handle commit failures separately.
6. Bug: `conn.close()` is never called on the exception path. Fix: always call `conn.close()` in both branches, or restructure with a `try/finally` inside `__exit__`: `try: rollback/commit finally: conn.close()`.

❓ CHECKPOINT QUESTION

If a `ValueError` is raised inside a `with SQLiteConnection(...) as cursor:` block, walk through every step that happens from the moment the exception is raised to when it reaches the caller's `try/except`.

CONTEXTLIB.CONTEXTMANAGER — THE GENERATOR-BASED APPROACH

ChatGPT • AI Assistant

A vertical split diagram showing a generator function body mapped to context manager protocol. Left side: the raw generator function code with four labeled regions. Region 1 (blue background): 'Code before yield — this is `__enter__`'. Region 2 (yellow): 'yield value — this is what gets bound to as variable'. Region 3 (green, inside try/finally): 'Code after yield in finally — this is `__exit__` success path'. Region 4 (red, inside except): 'except block — handles exception thrown at yield point'. Right side: a matching protocol diagram with `__enter__` and `__exit__` boxes, arrows connecting each region to its protocol method. Center dividing line labeled 'yield is the context boundary'. At the bottom, a warning box: 'Without try/finally: exception thrown at yield skips all code after yield — resource leaks!' Learning objective: See the exact correspondence between generator structure and the `__enter__`/`__exit__` protocol.

@contextmanager Execution Model — How yield Splits `__enter__` and `__exit__`

Student

Writing a whole class with `__enter__` and `__exit__` feels like a lot of boilerplate for something simple. Is there a shorter way?

ChatGPT • AI Assistant

CONCEPT CARD

Yes — `contextlib.contextmanager` is a decorator that turns a **generator function** into a context manager. Instead of a class with two methods, you write a function with a single `yield`. Code before the `yield` is your `__enter__`, code after the `yield` is your `__exit__`. The `yield` itself is what gets bound to the `as` variable.

EXAMPLE CARD

Example 1 — Basic: Rewriting ManagedFile with `@contextmanager`

python — Code Example

```
from contextlib import contextmanager

@contextmanager
def managed_file(filepath, mode='r'):
    print(f'[ENTER] Opening {filepath}')
    f = open(filepath, mode)
    try:
        yield f          # ← __enter__ returns f; body of 'with'
    finally:
        print(f'[EXIT] Closing {filepath}')
```

runs HERE

```
f.close()          # ← __exit__ logic: always runs

# Usage is IDENTICAL to the class version:
with managed_file('test.txt', 'w') as f:
    f.write('Hello!')

# Output:
# [ENTER] Opening test.txt
# [EXIT] Closing test.txt
```

Student

Why is the `try/finally` essential here? What happens if I remove it?

ChatGPT • AI Assistant

This is crucial. Without `try/finally`, if an exception occurs in the `with` block, `@contextmanager` throws that exception INTO the generator at the `yield` point. If you don't catch it, the generator terminates without running the cleanup code after `yield`. The file stays open. The `try/finally` guarantees the cleanup code runs regardless — the `finally` block runs whether the yield completed normally or an exception was thrown into it.

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — missing try/finally:

python — Code Example

```
@contextmanager
def managed_file(filepath):
    f = open(filepath)
    yield f          # If an exception occurs in the with block...
    f.close()        # ...this line NEVER runs! Resource leaked!
```

✓ Correct — always use try/finally:

python — Code Example

```
@contextmanager
def managed_file(filepath):
    f = open(filepath)
    try:
        yield f
```

finally:`f.close()` *# ALWAYS runs – exception or not***EXAMPLE CARD**

Example 2 — Intermediate: Rewriting the DB context manager with
 @contextmanager

python – Code Example

```

from contextlib import contextmanager
import sqlite3

@contextmanager
def sqlite_transaction(db_path):
    """
    @contextmanager version of the DB context manager.
    Same behavior: commit on success, rollback on exception.
    """
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    try:
        yield cursor                # Body of 'with' runs here – caller
        gets cursor
        conn.commit()              # Runs ONLY if body completed without
        exception
        print('[COMMIT] Transaction committed')
    except Exception as e:
        conn.rollback()            # Runs if body raised any exception
        print(f'[ROLLBACK] {type(e).__name__}: {e}')
        raise                      # Re-raise so caller sees the
        exception!
    finally:
        cursor.close()
        conn.close()              # ALWAYS runs – commit, rollback, or
        other
        print('[DB] Connection closed')

# Usage:
with sqlite_transaction(':memory:') as cur:
    cur.execute('CREATE TABLE items (id INT, name TEXT)')
    cur.execute("INSERT INTO items VALUES (1, 'Widget')")

# Output:

```

```
# [COMMIT] Transaction committed
# [DB] Connection closed
```

Student

In the exception branch, you do `raise` after the rollback. Why? Wouldn't the exception propagate automatically?

ChatGPT • AI Assistant

Great catch! In a `@contextmanager`, if you catch an exception in the `except` block and don't re-raise it, it's **suppressed** — the `with` block appears to complete normally. This is the opposite of a class-based `__exit__` where you return `True` to suppress. With `@contextmanager`, **silence is suppression** — you must explicitly `raise` to propagate. Forgetting `raise` after a rollback is one of the most common bugs in generator-based context managers.

EXAMPLE CARD

Example 3 — Practical: Temporary directory context manager

python — Code Example

```
import os
import shutil
import tempfile
from contextlib import contextmanager

@contextmanager
def temporary_directory(prefix='tmp_'):
    """
    Creates a temporary directory, yields its path,
    and ALWAYS deletes it on exit — even on exception.
    """
    tmpdir = tempfile.mkdtemp(prefix=prefix)
    print(f'[TMPDIR] Created: {tmpdir}')
    try:
        yield tmpdir          # Caller gets the directory path
    finally:
        shutil.rmtree(tmpdir, ignore_errors=True) # Always clean up!
        print(f'[TMPDIR] Deleted: {tmpdir}')

# Usage:
with temporary_directory(prefix='report_') as tmpdir:
```

```

# Work inside the temp directory
output_file = os.path.join(tmpdir, 'output.csv')
with open(output_file, 'w') as f:
    f.write('id,name\n1,Alice\n2,Bob')
print(f'Wrote to {output_file}')
# Process the file...

# After the block:
# [TMPDIR] Deleted: /tmp/report_abc123
# Directory and ALL contents are gone – no disk leaks

# Even if an exception occurs:
try:
    with temporary_directory() as tmpdir:
        raise RuntimeError('Processing failed!')
except RuntimeError:
    pass

# [TMPDIR] Deleted: /tmp/tmp_xyz789
# Directory still cleaned up!

```

EXAMPLE CARD**Example 4 — Advanced: Context manager that measures execution time**

python — Code Example

```

import time
from contextlib import contextmanager

@contextmanager
def timer(label='Block'):
    """
    Measures and prints execution time of any with block.
    Works even if an exception is raised.
    """
    start = time.perf_counter()
    try:
        yield
    finally:
        elapsed = time.perf_counter() - start
        print(f'[TIMER] {label}: {elapsed:.4f}s')

# Basic usage:
with timer('Data processing'):
    time.sleep(0.1)
    result = sum(range(1_000_000))
# [TIMER] Data processing: 0.1423s

```

```
# Also works with exceptions – timing is still reported:
try:
    with timer('Failing operation'):
        time.sleep(0.05)
        raise ValueError('Something broke')
except ValueError:
    pass
# [TIMER] Failing operation: 0.0512s
# (Then ValueError propagates)
```

INTERVIEW CARD

- **Frequency:** High — a favorite in Python-focused interviews
- **Question:** 'Rewrite this class-based context manager using `@contextmanager` . What is the rule about `try/finally` ?'
- **Expected Answer:** Use `@contextmanager` from `contextlib` . Code before `yield` = `__enter__` . Code after `yield` (in `finally`) = `__exit__` . `try/finally` is mandatory because exceptions are thrown into the generator at the yield point — without `finally` , cleanup code after `yield` is skipped on exceptions.
- **Follow-up:** 'In a `@contextmanager` , how do you suppress an exception vs propagate it?'

EXERCISE

Practice Problem: Write a `@contextmanager` called `suppress_and_log(exc_type)` that:

1. Runs the `with` block normally
2. If the specified exception type is raised, logs `'Suppressed: <exc_type>: <message>'` and suppresses it
3. All other exceptions propagate normally

python – Code Example

```
@contextmanager
def suppress_and_log(exc_type):
    # TODO
    pass

with suppress_and_log(ValueError):
    raise ValueError('bad input') # Should be suppressed and logged
print('Continues here')          # Should print
```

```
with suppress_and_log(ValueError):  
    raise TypeError('wrong type') # Should propagate – not suppressed
```

🌟 AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

"I understand both class-based and @contextmanager approaches in Python. Now show me 8 real-world context manager patterns: database transaction, distributed lock with Redis, HTTP session pooling, performance profiling, audit logging, feature flag toggle, mock patching, and thread-local context injection. For each: (1) explain the business problem, (2) show the @contextmanager implementation, (3) show usage, (4) ask me what happens if I forget try/finally, (5) give one interview question."

🔗 PRACTICE QUIZ

Concept Questions:

1. In a `@contextmanager` function, what happens when an exception occurs in the `with` block — where does the exception go?
2. In a `@contextmanager`, what is the difference between catching an exception and re-raising it vs catching and NOT re-raising?
3. Why is `yield` inside `@contextmanager` limited to exactly ONE yield?

Output Prediction Questions:

1. What does this print?

python – Code Example

```
from contextlib import contextmanager  
  
@contextmanager  
def ctx():  
    print('ENTER')  
    try:  
        yield 42  
    finally:  
        print('EXIT')  
  
with ctx() as val:  
    print(f'GOT: {val}')
```

```
raise ValueError('oops')
```

1. What is the output if the `raise` in the body is changed to `pass` ?

Debugging Question:

1. This context manager is supposed to suppress `FileNotFoundError` but it propagates anyway — find the bug:

python — Code Example

```
@contextmanager
def safe_open(path):
    try:
        f = open(path)
        yield f
        f.close()
    except FileNotFoundError as e:
        print(f'File not found: {e}')
        # Bug: missing raise suppression logic
        raise # This re-raises – defeats the purpose!
```

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. The exception is thrown INTO the generator at the yield point using `generator.throw(exc_type, exc_val, exc_tb)`. If the generator has a `try/except` around the yield, it can catch it. If not, the generator terminates and the exception propagates.
2. Catching and re-raising → exception propagates to the caller (like returning `False` from `__exit__`). Catching and NOT re-raising → exception is suppressed, execution continues after the `with` block normally (like returning `True` from `__exit__`).
3. A generator used as a context manager must yield exactly once. Yielding twice would mean two `__enter__` phases, which the protocol doesn't support. `@contextmanager` raises `RuntimeError: generator didn't stop` if no yield occurs, or `RuntimeError: generator didn't stop after throw()` if more than one.
4. Prints: `ENTER`, `GOT: 42`, `EXIT`, then `ValueError: oops` propagates.
5. Without the raise: Prints `ENTER`, `GOT: 42`, `EXIT`. No exception.
6. The `raise` at the end re-raises `FileNotFoundError` — negating the suppression. To suppress, remove the `raise`. But also: `f.close()` after `yield`

should be in a `finally` block, not inline — or the close is skipped if an exception occurs inside the `with` body.

❓ CHECKPOINT QUESTION

You write `@contextmanager` for a lock that must always be released. Where exactly should `lock.release()` go — in `except`, in `finally`, or after the `yield` with no try block — and why?

PRACTICE PROBLEMS — SUPPRESS EXCEPTION AND EXECUTION TIMER

ChatGPT • AI Assistant

A side-by-side diagram comparing both practice problem context managers. Left panel: 'suppress_exception'. Shows `__enter__` box: 'returns self — no state needed'. Body box: 'with block runs'. Two `__exit__` paths: green path (target exception type) → 'issubclass check → True → exception suppressed → execution continues'. Red path (other exception) → 'return False → exception propagates'. Small inheritance diagram shows: `OSError` → `FileNotFoundError`, `PermissionError`; note: 'issubclass catches all children'. Right panel: 'execution_timer'. Shows `__enter__` box: 'self.start = perf_counter(); return self'. Body box with clock icon: 'with block runs — time passes'. `__exit__` box: 'elapsed = perf_counter() - self.start; print(elapsed); return False'. Post-block arrow: 't.elapsed still accessible because t = the instance'. Both panels share a bottom note: '`__exit__` always called — even on exception. This is the guarantee.' Learning objective: Contrast stateless suppression vs stateful timing.

suppress_exception vs execution_timer — State and Exception Flow Comparison

Student

You mentioned two practice problems from the topic spec. Can we work through both of them properly?

ChatGPT • AI Assistant

🔥 EXERCISE

Practice Problem 1 — `suppress_exception(exc_type)` context manager

Write BOTH a class-based AND a `@contextmanager` version. They should behave identically:

python — Code Example

```
# ——— VERSION A: Class-based ———
class suppress_exception:
```

```

Suppresses a specific exception type.
All other exceptions propagate.
"""

def __init__(self, exc_type):
    self.exc_type = exc_type

def __enter__(self):
    return self    # No resource to return

def __exit__(self, exc_type, exc_val, exc_tb):
    # TODO:
    # If exc_type matches self.exc_type (or is a subclass), return
    True
    # Otherwise return False
    pass

# — VERSION B: @contextmanager —————
from contextlib import contextmanager

@contextmanager
def suppress_exception_cm(exc_type):
    # TODO:
    # try: yield
    # except the target type: pass (suppress it)
    # other exceptions propagate automatically
    pass

# — TEST BOTH —————
for cm in [suppress_exception(ValueError),
suppress_exception_cm(ValueError)]:
    with cm:
        raise ValueError('expected error')    # Suppressed
        print('Continued after ValueError')    # Should print

for cm in [suppress_exception(ValueError),
suppress_exception_cm(ValueError)]:
    try:
        with cm:
            raise TypeError('unexpected')    # Should propagate
    except TypeError:
        print('TypeError propagated correctly')

```

Solution:

python – Code Example

```

class suppress_exception:
    def __init__(self, exc_type):
        self.exc_type = exc_type

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        # issubclass check handles inheritance: suppress
        FileNotFoundError
        # also suppresses subclasses of FileNotFoundError
        return exc_type is not None and isinstance(exc_type,
self.exc_type)

@contextmanager
def suppress_exception_cm(exc_type):
    try:
        yield
    except exc_type:
        pass # Suppress it – don't re-raise
        # Any other exception type propagates automatically

```

Student

The class version uses `issubclass` — why?

ChatGPT • AI Assistant

Because Python's exception hierarchy uses inheritance. `FileNotFoundError` is a subclass of `OSError`. If someone calls `suppress_exception(OSError)`, they probably want to suppress `FileNotFoundError` too — just like `except OSError` in a regular try block catches all subclasses. Using `issubclass` mirrors Python's own exception handling behavior.

🔥 EXERCISE

Practice Problem 2 — `execution_timer` context manager

python – Code Example

```

import time
from contextlib import contextmanager

```

```

# ——— VERSION A: Class-based —————
class execution_timer:
    """
    Measures execution time of a with block.
    Prints elapsed time on exit.
    Stores elapsed time as .elapsed attribute.
    Does NOT suppress exceptions.
    """

    def __init__(self, label='Block', precision=4):
        self.label = label
        self.precision = precision
        self.elapsed = None
        self._start = None

    def __enter__(self):
        self._start = time.perf_counter()
        return self  # Return self so caller can access .elapsed
                    # after block

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.perf_counter() - self._start
        print(f'[TIMER] {self.label}: {self.elapsed:.{self.precision}f}s')
        return False  # Never suppress exceptions

# ——— VERSION B: @contextmanager —————
@contextmanager
def execution_timer_cm(label='Block'):
    start = time.perf_counter()
    try:
        yield
    finally:
        elapsed = time.perf_counter() - start
        print(f'[TIMER] {label}: {elapsed:.4f}s')

# ——— USAGE —————

# Class version – can inspect elapsed after block:
with execution_timer('Database query', precision=3) as t:
    time.sleep(0.05)
    result = sum(range(100_000))
print(f'Saved elapsed: {t.elapsed:.3f}s')  # Access t.elapsed after
block!
# [TIMER] Database query: 0.050s
# Saved elapsed: 0.050s

```

```
# Generator version – simpler but can't access elapsed externally:
with execution_timer_cm('File processing'):
    time.sleep(0.03)
# [TIMER] File processing: 0.0301s

# Both work correctly even when an exception occurs:
try:
    with execution_timer('Failing task') as t:
        time.sleep(0.02)
        raise RuntimeError('Task failed')
except RuntimeError:
    print(f'Failed after {t.elapsed:.3f}s')
# [TIMER] Failing task: 0.020s
# Failed after 0.020s
```

Student

The class version lets you access `t.elapsed` AFTER the block — that's a real advantage of class-based over `@contextmanager` !

ChatGPT • AI Assistant

Exactly — that's one of the key reasons to choose class-based: **state that persists after the block**. The `as t` variable gives you the context manager object itself, so any attributes set in `__exit__` are accessible after the `with` block exits. With a `@contextmanager`, the generator's local variables are gone after it finishes. If you need to inspect metrics after the block, use the class approach.

🌐 REAL WORLD APPLICATION

Production uses of `execution_timer` :

- Performance benchmarking in CI/CD pipelines
- Slow query detection in ORMs (SQLAlchemy's event hooks use this pattern)
- API endpoint response time measurement
- Data pipeline stage profiling

⚠️ COMMON PITFALL / MISTAKE

✗ Incorrect — trying to access `elapsed` from a `@contextmanager` version:

python – Code Example

```
with execution_timer_cm('Query') as t:
    do_work()
print(t.elapsed) # AttributeError! t is None – yield returned None
```

✓ Correct — use class version when you need post-block data:

python – Code Example

```
with execution_timer('Query') as t:
    do_work()
print(t.elapsed) # Works! t is the execution_timer instance
```

INTERVIEW CARD

- **Frequency:** Medium — popular in interviews for DevOps, backend, and data pipeline roles
- **Question:** 'Write a context manager that measures and stores the execution time of a code block. The elapsed time should be accessible after the block exits.'
- **Expected Answer:** Class-based context manager with `_start` recorded in `__enter__`. `elapsed` computed and stored in `__exit__`. Return `self` from `__enter__` so the `as` variable gives access to the object post-block.
- **Follow-up:** 'What is the advantage of returning `self` from `__enter__` vs returning a new object?'

AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

"I've mastered basic context managers in Python. Now challenge me with 6 advanced context manager patterns used in real frameworks: SQLAlchemy's session scope, unittest.mock.patch as a context manager, threading.Lock as a context manager, contextlib.ExitStack for dynamic resource management, contextlib.AsyncContextManager for async/await, and a re-entrant context manager. For each: (1) explain what problem it solves, (2) show a simplified implementation from scratch, (3) compare it to the real stdlib/framework version, (4) give me an interview question. Do not continue until I answer."

PRACTICE QUIZ

Concept Questions:

1. What is the key advantage of a class-based context manager over `@contextmanager` when you need post-block data?
2. In `suppress_exception.__exit__`, why use `issubclass(exc_type, self.exc_type)` instead of `exc_type == self.exc_type`?
3. In `execution_timer`, why is `time.perf_counter()` preferred over `time.time()`?

Output Prediction Questions:

1. What does this print?

python – Code Example

```
with execution_timer('Test') as t:
    time.sleep(0.1)
print(type(t))
```

1. What does this print?

python – Code Example

```
with suppress_exception(OSError):
    raise FileNotFoundError('no file') # FileNotFoundError is a
    subclass of OSError
print('continued')
```

Debugging Question:

1. This timer context manager shows `0.0000s` for a block that takes 2 seconds — find the bug:

python – Code Example

```
class BrokenTimer:
    def __enter__(self):
        return self
    def __exit__(self, *args):
        elapsed = time.perf_counter() - self.start # Bug!
        print(f'Elapsed: {elapsed:.4f}s')
        return False
```

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Class-based: `__enter__` returns `self`, so the `as` variable points to the context manager object. Attributes set during `__exit__` persist and are accessible after the `with` block. With `@contextmanager`, local variables in the generator are not accessible post-yield.
2. `issubclass` handles Python's exception inheritance hierarchy. `exc_type == self.exc_type` would only suppress the exact class, missing all subclasses. Real exception handling always catches subclasses.
3. `time.perf_counter()` is a high-resolution monotonic clock — it measures wall-clock time precisely and is not affected by system time adjustments.
`time.time()` can jump backwards if the system clock is corrected (NTP sync, DST), causing negative elapsed times.
4. Prints `<class '__main__.execution_timer'>` — `t` is the context manager instance itself.
5. Prints `'continued'` — `FileNotFoundError` is a subclass of `OSError`, so `issubclass(FileNotFoundError, OSError)` is `True`, and the exception is suppressed.
6. Bug: `self.start` was never set. `__enter__` returns `self` but never records the start time. Fix: add `self.start = time.perf_counter()` inside `__enter__`.

SUMMARY CARD

- `__enter__` = setup; `__exit__` = teardown — `__exit__` ALWAYS runs
- `__exit__` receives `(exc_type, exc_val, exc_tb)` — all `None` if no exception
- Return `True` from `__exit__` to suppress; `False` / `None` to propagate
- `@contextmanager`: code before `yield` = enter; code in `finally` after `yield` = exit
- `try/finally` is MANDATORY in `@contextmanager` — without it, exceptions skip cleanup
- In `@contextmanager`, don't re-raise to suppress; re-raise (or no except) to propagate
- Choose class-based when you need state accessible after the block; `@contextmanager` for simplicity
- `issubclass` is the correct check for exception type matching — mirrors Python's own `except` behavior

❓ CHECKPOINT QUESTION

You need a context manager that (1) suppresses only `ValueError`, (2) logs all other exceptions without suppressing them, AND (3) exposes whether suppression

happened via a `.suppressed` attribute after the block. Which approach — class-based or `@contextmanager` — is better here, and why?

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including context managers, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Context Manager Protocol — The with Statement Execution Flow:** A vertical flowchart showing the full with-statement lifecycle. Top box: 'with M...